

Weighted Graph Clustering via Scale Contraction and Graph Structure Learning

Haobing Liu
Ocean University of China
Qingdao, China
haobingliu@ouc.edu.cn

Yinuo Zhang
Ocean University of China
Qingdao, China
zyn6261@stu.ouc.edu.cn

Tingting Wang
Ocean University of China
Qingdao, China
wtt2022@stu.ouc.edu.cn

Ruobing Jiang*
Ocean University of China
Qingdao, China
jrb@ouc.edu.cn

Yanwei Yu*
Ocean University of China
Qingdao, China
yuyanwei@ouc.edu.cn

Abstract

Graph clustering aims to partition nodes into distinct clusters based on their similarity, thereby revealing relationships among nodes. Nevertheless, most existing methods do not fully utilize these edge weights. Leveraging edge weights in graph clustering tasks faces two critical challenges. (1) The introduction of edge weights may significantly increase storage space and training time, making it essential to reduce the graph scale while preserving nodes that are beneficial for the clustering task. (2) Edge weight information may inherently contain noise that negatively impacts clustering results. However, few studies can jointly optimize clustering and edge weights, which is crucial for mitigating the negative impact of noisy edges on clustering task. To address these challenges, we propose a contractile edge-weight-aware graph clustering network. Specifically, a cluster-oriented graph contraction module is designed to reduce the graph scale while preserving important nodes. An edge-weight-aware attention network is designed to identify and weaken noisy connections. In this way, we can more easily identify and mitigate the impact of noisy edges during the clustering process, thus enhancing clustering effectiveness. We conducted extensive experiments on three real-world weighted graph datasets. In particular, our model outperforms the best baseline, demonstrating its superior performance. Furthermore, experiments also show that the proposed graph contraction module can significantly reduce training time and storage space.

CCS Concepts

• **Computing methodologies** → **Unsupervised learning; Neural networks.**

Keywords

Graph Clustering, Graph Attention Network, Graph Structure Learning, Graph Contraction

*Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792363>

ACM Reference Format:

Haobing Liu, Yinuo Zhang, Tingting Wang, Ruobing Jiang, and Yanwei Yu. 2026. Weighted Graph Clustering via Scale Contraction and Graph Structure Learning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792363>

1 Introduction

Graph clustering [22, 36], a specialized form of clustering, aims to partition nodes within a graph into distinct clusters based on their connection patterns. Its primary objective is to ensure high similarity among nodes within the same cluster while minimizing similarity between nodes in different clusters. Graph clustering plays a crucial role in uncovering relationships among nodes in a graph. It has numerous applications in both academia and industry, including community detection [30], image recognition [42], and recommendation systems [8].

Existing deep graph clustering methods can be categorized into two classes based on their network architecture: graph neural network (GNN)-based and multi-layer perceptron (MLP)-based methods. MLP-based methods [21, 37] utilize MLPs to extract informative features from graph data. But they struggle to effectively capture non-euclidean structural information. Recent progress in the Web domain has further validated the role of structural integration in deep clustering [6] and the use of advanced graph representations [7] in facilitating the capture of intricate user patterns. In recent years, non-Euclidean graph data has become increasingly pervasive, as it encapsulates not only node attributes but also topological structure information, which characterizes the relationships among data points [24]. To fully leverage this information, GNN-based methods [4, 17] employ GNN encoders (e.g., Graph Convolutional Network (GCN) [19], Graph Attention Network (GAT) [32]) to model graph data. These methods propagate information by aggregating features from neighboring nodes and learning low-dimensional representations, which brings nodes within the same cluster closer in the embedding space. Although some methods can handle weighted graphs (e.g., by introducing adjacency matrices with edge weights), almost all of them neglect that edge weight information often contains noise.

Using edge weights effectively in graph clustering faces two critical challenges. *Firstly*, the introduction of edge weights significantly increases resource cost. Tuples are often chosen to store the graph

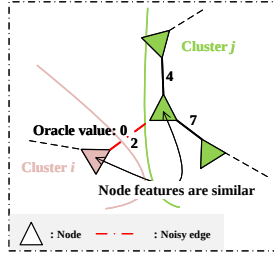


Figure 1: An illustrative example of graph clustering on a noisy weighted graph.

structure due to the requirement of scalability. If edge weights are not considered, 2-tuples (i.e., (node, node)) can be leveraged. While edge weights are taken into account, 3-tuples (i.e., (node, node, weight)) should be utilized. This leads to a 50% increase in GPU memory usage during the model training process. Furthermore, this may also increase the time cost. To mitigate this issue, subgraph sampling methods [40, 44] are adopted to extract a subgraph from the original graph, thereby reducing costs. However, existing sampling strategies often randomly select nodes or edges, which can lead to subgraphs that are not sufficiently representative of the original graph’s clustering structure. Random sampling may exclude nodes that are densely connected within a cluster (they are pivotal for defining internal cohesion). If density is considered during sampling, nodes in low-density regions may be overlooked, causing the number of clusters in the subgraph to be lower than the true value. Therefore, a critical challenge lies in *how to reduce the scale of weighted graphs while preserving nodes that contribute significantly to the clustering structure*.

Secondly, graph structures often contain noisy edges [34, 38], indicating a discrepancy between the edge weights and their corresponding oracle values. Such noisy edge weights may lead target nodes to aggregate more features from unrelated neighbors, thereby leading to incorrect clustering results. As shown in Figure 1, node features of the green and red nodes are similar, and they are connected by an edge with a weight of 2. Existing methods might cluster these two nodes into the same group. In reality, these two nodes belong to distinct clusters. Some methods [14, 31] attempt to mitigate the impact of noisy edges by optimizing adjacency matrices. However, these methods rarely optimize graph structure with clustering together. Consequently, a critical challenge is *how to jointly optimize clustering and edge weights, thereby removing noisy edges that harm clustering task*.

To address these challenges, we propose a contractile edge-weight-aware graph clustering network (CeeGCN in short). To solve the *first challenge*, we design a cluster-oriented graph contraction module. This module extracts a subgraph from the original graph by jointly considering node density, distance, and similarity. The extraction process preserves the intrinsic clustering structure of the original graph. To tackle the *second challenge*, we design an edge-weight-aware sparse graph attention network (EWSGAT in short). Compared to GAT, EWSGAT explicitly integrates both node features and edge weights when computing attention coefficients. It then utilizes an α -entmax function [26] for normalization,

which automatically sparsifies attention coefficients by effectively setting the weights of noisy edges to zero, thereby suppressing the influence of noisy edges. Based on sparse attention coefficients, we can optimize the graph structure, thereby improving clustering performance.

For clarity, the main contributions of this paper are summarized as follows:

- We design a cluster-oriented graph contraction module that significantly reduces resource consumption during training while ensuring clustering performance. This lays the foundation for future solutions to large-scale weighted graph clustering problems.
- We develop an edge-weight-aware sparse graph attention network that reduces the impact of noisy edges and can optimize the graph structure. This ensures our model with robustness and scalability.
- We construct three weighted graph datasets in the real world. We conduct extensive experiments on these datasets. The experimental results demonstrate that the proposed model (i.e., CeeGCN) outperforms competitive methods. In particular, CeeGCN with an average Micro-F1 improvement of 31.5% compared with the best baseline.

2 Related Work

2.1 GNN-based Graph Clustering

In recent years, graph clustering techniques have demonstrated promising performance [18]. In particular, with the help of GNNs, graph clustering methods can leverage both node features and graph structures. GNN-based methods utilize GNNs to obtain node representations by aggregating features from neighbors, subsequently clustering these representations. DGCluster [4] proposes a trainable modular method that models the graph structure using GNN (e.g., GCN, GAT) to learn node representations, thereby accomplishing cluster number estimation. Some methods introduce GCNs for learning node representations. For example, SSGCN [14] employs a graph structure learning layer and an adaptive graph convolutional layer to jointly optimize both graph structure and node representations. DyFSS [43] assigns each self-supervised learning (SSL) task to a task-specific GCN layer to extract features using the corresponding SSL loss.

Some methods introduce GATs, empowering models to dynamically learn the importance of neighbors. NCAGC [35] introduces a neighborhood contrastive mechanism to directly construct positive and negative sample pairs within the node representation space generated by a GAT encoder. DNENC [33] combines a dual-branch autoencoder to simultaneously reconstruct both structural and attribute information. EFR-DGC [12] utilizes a GAT to construct a graph autoencoder to extract the topological relationships of nodes. This information is then combined with hierarchical attribute information to generate enhanced node representations that are beneficial for clustering.

Additionally, some methods [16] integrate GCNs with autoencoders to fully exploit node features. SCGC [20], based on graph autoencoders, aims to improve the efficiency and scalability of existing deep graph clustering methods by decoupling transformation and propagation operations.

2.2 Graph Sampling and Graph Structure Learning

Existing graph sampling methods fall into three classes based on their sampling granularity: node sampling, layer-wise sampling, and subgraph sampling [40, 44]. Node sampling methods [29, 41] randomly select a subset of nodes from a target node's neighborhood for message passing. Layer-wise sampling methods [2, 44] divide the graph into multiple layers, then perform global sampling at each message passing layer. However, both classes of methods require resampling before each forward pass. Subgraph sampling methods [39, 40] sample relatively independent connected subgraphs from the original graph, training each subgraph independently as a batch. Yet current random selection strategies often compromises global structure and fails to preserve critical edges.

The core idea of graph structure learning is to jointly optimize the graph structure itself along with node representations. VIB-GSL [31] generates an information bottleneck graph through feature masking and graph structure learning and then use GNNs to encode this graph to obtain node representation. SSGCN [14] introduces a graph structure learning layer before each graph convolution layer, exploring end-to-end joint graph structure and representation learning. However, these methods overlook edge weights and can not jointly optimize graph structure and clustering. Specifically, since nodes connect to both homophilous (class-consistent) and heterophilous (class-divergent) neighbors [28], the lack of refined modeling of edge weights hinders effectively suppress noise introduced by heterophilous neighbors during aggregation process.

3 Preliminaries

Weighted Graph: A weighted graph is defined as $G = (V, E, \mathcal{W})$, $V = \{v_1, v_2, \dots, v_n\}$ is the set of node, where n is the number of nodes, and E is the set of edges. $\mathcal{W}$ is the weight matrix. In an undirected weighted graph, if there is an edge between v_i and v_j , $w_{ij} \neq 0$; otherwise, $w_{ij} = w_{ji} = 0$.

Problem Formulation: Given a weighted graph $G = (V, E, \mathcal{W})$, the goal of graph clustering is to partition the n nodes in the graph into K disjoint clusters $C = \{C_1, C_2, \dots, C_K\}$ so that each node $v_i \in V$ is assigned a cluster label $c_i \in \{1, 2, \dots, K\}$ indicating its membership. $\forall k, k', C_k \cap C_{k'} = \emptyset$.

Core nodes: Core nodes are a small set of representative nodes jointly selected based on two criteria: local density and distance between nodes. These nodes have high local density and cover a wide range of the original graph. We will introduce the details of the selection in Section 4.1.

Subgraph: The subgraph G' is a subset of nodes and edges selected from the original graph. In this paper, a cluster-oriented graph contraction module is designed to get G' . Specifically, core nodes are first selected; then the remaining nodes are selected based on their importance relative to the core nodes. These nodes and the edges among them form G' .

4 The Proposed Method

The overall architecture of our proposed model (i.e., CeeGCN) is shown in Figure 2. The model consists of three modules: (a) Cluster-oriented graph contraction module. This module first selects core nodes and important nodes around the core nodes, and constructs

a subgraph. (b) EWSGAT. This module calculates attention coefficients by considering both node features and edge weights. Noise edges are suppressed via the α -entmax function. (c) Inference and training objective. The training process is jointly guided by a graph structure-based loss and a modularity-based loss.

4.1 Cluster-oriented Graph Contracting

As shown in Figure 2(a), we propose a cluster-oriented graph contraction module that selects core nodes by combining node density and distance, and the remaining nodes are selected based on their importance relative to the core nodes.

Firstly, we introduce the selection of core nodes. High-density nodes typically reside at the core of clusters. Meanwhile, core nodes should maintain appropriate spacing to maximize coverage of the entire graph.

We estimate the density ρ_i of node i : $\rho_i = \sum_{j \in \mathcal{N}_i} w_{ij}$, where w_{ij} denotes the weight between node i and node j , and $\mathcal{N}_i$ denotes the set of neighbors of node i . The node with the highest density is selected as the initial core node.

Assume that we should select O core nodes. To determine the remaining $O - 1$ core nodes, we estimate the distance between node i and selected core nodes: $\theta_i = \sum_j \text{dist}(v_i, v_j)$, where v_j represents a selected core node; $\text{dist}(v_i, v_j)$ denotes the shortest path length from node i to node j , calculated using the Dijkstra algorithm. Let $\mathcal{R}_{\rho_i} = \text{rank}(\rho_i)$ and $\mathcal{R}_{\theta_i} = \text{rank}(\theta_i)$ denote the scores obtained by sorting the respective values in ascending order, where nodes with earlier-ranked positions are assigned higher scores. Then comprehensive scores are obtained through the weighted integration of $\mathcal{R}_{\rho_i}$ and $\mathcal{R}_{\theta_i}$:

$$\mathcal{R}_i = \varepsilon \mathcal{R}_{\rho_i} + (1 - \varepsilon) \mathcal{R}_{\theta_i}, \quad (1)$$

where ε is a hyperparameter that balances the influence of density and distance. The node with the highest comprehensive score is selected as a core node. This selection process is iterated a total of $O - 1$ times.

Secondly, we introduce the selection of remaining nodes. We calculate the importance scores from each core node to other nodes via the personalized PageRank algorithm [11]:

$$S = \varphi \cdot (\mathcal{I} - (1 - \varphi) \cdot \mathcal{W}\mathcal{D}^{-1}), \quad (2)$$

where $\mathcal{I}$ is the identity matrix, $\mathcal{D}$ is the degree matrix and φ is a tunable hyperparameter. Based on S_i (where i denotes the core node i), nodes with important scores greater than a threshold are selected.

All selected nodes and the edges among them form a subgraph G' , which captures the original graph's clustering structure.

4.2 Edge-weight-aware Sparse Graph Structure Learning

Vanilla GAT computes attention coefficients solely based on node features, neglecting valuable edge weight information. In this paper, we propose an edge-weight-aware sparse graph attention network (i.e., EWSGAT), as shown in Figure 2(b), which explicitly incorporates edge weights when calculating attention coefficients. The attention coefficient is redefined as follows:

$$e'_{iz} = f_{iz} + e_{iz}, \quad z \in \mathcal{N}_i \cup v_i \quad (3)$$

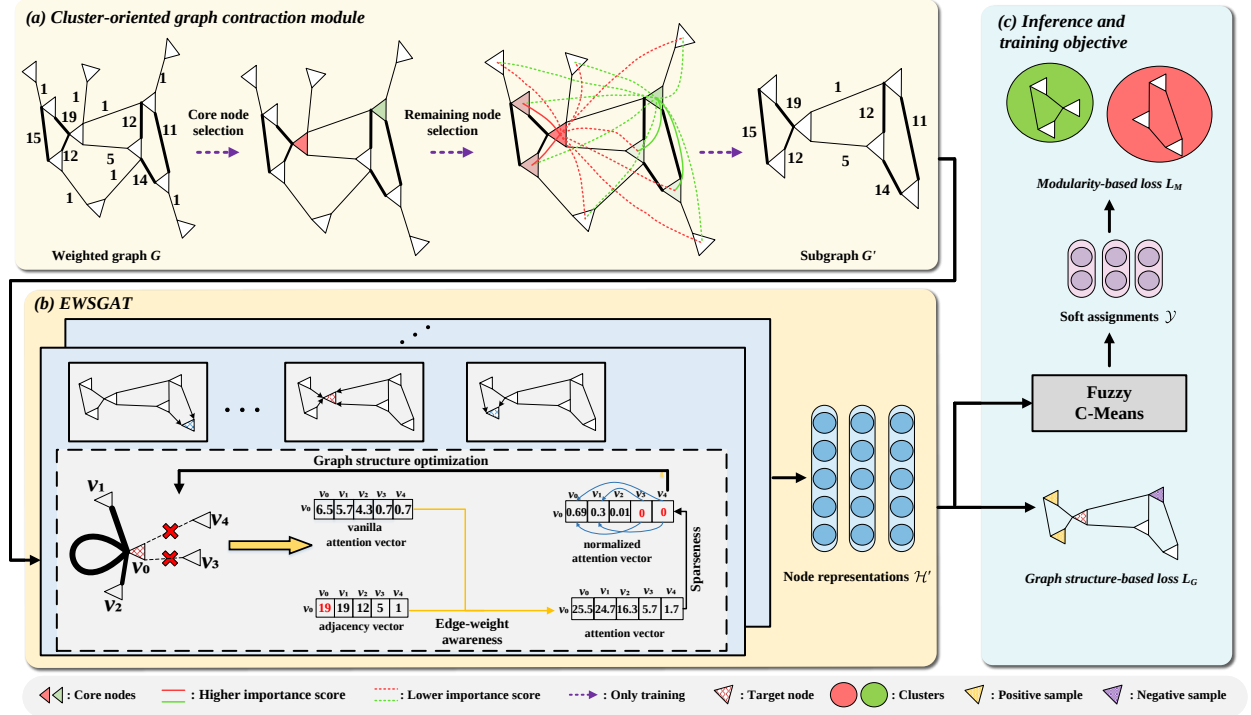


Figure 2: The architecture of CeeGCN, including (a) Cluster-oriented graph contraction module, (b) EWSGAT, and (c) Inference and training objective.

where e_{iz} denotes the vanilla attention coefficients; f_{iz} is the impact factor that is derived from edge weights; $\mathcal{N}_i$ denotes the set of neighbors for node i . e_{iz} is calculated as follows:

$$e_{iz} = (W_1 \mathbf{h}_i)^\top \cdot (W_1 \mathbf{h}_z), \quad (4)$$

where W_1 denotes a learnable weight matrix. $\mathbf{h}_i$ and $\mathbf{h}_z$ are node representations of node i and node z , respectively, which serve as the input to the EWSGAT layer. f_{iz} is calculated as follows:

$$f_{iz} = \frac{w_{iz}}{\sum_{j \in \mathcal{N}_i} w_{ij} + w_{ii}}, \quad (5)$$

where w_{ii} represents the self-loop weight of node i . $w_{ii} = \max \{w_{ij}\}$, $j \in \mathcal{N}_i$.

Compared with the vanilla calculation method, higher edge weights lead to larger attention coefficients. After normalization, these larger attention coefficients will be further significantly increased.

To normalize the attention coefficients, we further introduce the α -entmax function to replace the softmax function used in vanilla GAT. By incorporating the hyperparameter α , when $1 < \alpha < 2$, any attention score below a certain threshold is set to 0, focusing the attention on the most related neighbors and reducing the impact of noisy connections.

Formally, we collect the redefined attention coefficients for all neighbors of node i , denoted as $\mathbf{e}'_i = [e'_{i1}, e'_{i2}, \dots, e'_{ij}, \dots, e'_{i(|\mathcal{N}_i|+1)}]$, where $\mathcal{N}_i$ denotes the set of neighbors of node i ; $|\mathbf{e}'_i| = |\mathcal{N}_i \cup v_i| = |\mathcal{N}_i| + 1$. The normalized redefined attention coefficients between

node i and its neighbors are then obtained by:

$$\mathbf{a}'_i = \alpha\text{-entmax}(\mathbf{e}'_i) = [(\alpha - 1)\mathbf{e}'_i - \tau \mathbf{1}]_+^{\frac{1}{\alpha-1}}, \quad (6)$$

where $\mathbf{1}$ represents a vector of all ones, $[x]_+ = \max\{0, x\}$, and by $[x]_+$ its elementwise application to vectors. So $[(\alpha - 1)e'_{iz} - \tau]_+ = \max\{0, [(\alpha - 1)e'_{iz} - \tau]\}$. τ denotes the threshold for α -entmax. When the attention coefficient satisfies $e'_{iz} \leq \frac{\tau}{\alpha-1}$, the normalized probability becomes 0.

To ensure that all probabilities sum to 1, τ must satisfy:

$$\sum_j [(\alpha - 1)e'_{iz} - \tau(e'_{iz})]_+^{\frac{1}{\alpha-1}} = 1, \quad (7)$$

where $\tau(e'_{iz})$ denotes the dynamically computed the value of τ based on e'_{iz} . Considering that each node may have a large number of neighbors, we adopt the binary search method [5] to calculate τ . The detailed calculation process is provided in the Appendix.

After obtaining the normalized attention coefficients, the node representation can be updated using representations of its neighbors as follows:

$$\mathbf{h}'_i = \sigma\left(\sum_{z \in \mathcal{N}_i \cup v_i} \mathbf{a}'_{iz} W_2 \mathbf{h}_z\right), \quad (8)$$

where $\sigma(\cdot)$ is the ELU [9] function; $\mathbf{a}'_{iz}$ is the normalized attention coefficient between node i and node z ; W_2 is a learnable weight matrix; $\mathbf{h}'_i$ is the node representation of node i , which serve as the output of the EWSGAT layer.

Vanilla multi-head attention mechanism may neglect critical information from certain attention heads, so we introduce a learnable parameter γ_t to adaptively assign importance to head t . Assume

Algorithm 1: Training and inferencing of CeeGCN.

Input: the original graph G , the number of clusters K , and hyperparameters
Output: K disjoint clusters C

- 1 Select the first core node via node density: $\rho_i = \sum_{j \in \mathcal{N}_i} w_{ij}$;
- 2 **repeat**
- 3 Compute distance $\theta_i = \sum_j \text{dist}(v_i, v_j)$;
- 4 Select core node with the highest comprehensive score $\mathcal{R}_i$ based on density and distance;
- 5 **until** selecting $O - 1$ additional core nodes;
- 6 Calculate importance scores: $S = \varphi \cdot (I - (1 - \varphi) \cdot \mathcal{W}\mathcal{D}^{-1})$ and select the nodes with scores greater than a threshold as the remaining nodes;
- 7 Construct subgraph G' based on selected nodes;
- 8 Initialize the node representations matrix $\mathcal{H}$;
- 9 **repeat**
- 10 $\mathcal{H}' = \text{EWSGAT}(\mathcal{H})$;
- 11 $\mathcal{Y} = \text{FCM}(\mathcal{H}')$;
- 12 Assign clusters: $c_i = \underset{k \in \{1, 2, \dots, K\}}{\text{argmax}} y_{ik}$;
- 13 Updated edge weights: $w_{ij} \leftarrow \frac{a'_{ij} + a'_{ji}}{2} \cdot w_{ij}$;
- 14 Compute the total loss L via $L = L_G + \eta L_M$;
- 15 Backpropagate and update model parameters using L ;
- 16 **until** convergence criterion is satisfied;
- 17 Input the original graph G into the trained model to obtain the prediction result C .

that there are T heads. We fuse these heads to obtain the final node representation of node i as follows:

$$\mathbf{h}'_i = \sum_t \gamma_t \mathbf{h}'_{it}, \quad t = 1, 2, \dots, T, \quad (9)$$

where γ_t is a learnable parameter and $\mathbf{h}'_{it}$ can be calculated with Eq. (8). Finally, all node representations output by EWSGAT can be concatenated to form a node representation matrix $\mathcal{H}'$.

4.3 Inference and Training Objective

To obtain soft clustering assignments, we apply the Fuzzy C-Means (FCM) clustering algorithm [3] to $\mathcal{H}'$, as shown in Figure 2(c). Unlike the standard euclidean distance, our approach directly utilizes the inner product to characterize the relationship between nodes and cluster centers. Specifically, the optimization process is performed iteratively. In each iteration, we first update the membership probability $\mathcal{Y}_{ik}$ of node i belonging to cluster k via:

$$\mathcal{Y}_{ik} = \left[\sum_{j=1}^K \left(\frac{\mathbf{h}'_i{}^\top \boldsymbol{\mu}_k}{\mathbf{h}'_i{}^\top \boldsymbol{\mu}_j} \right) \right]^{-1}, \quad (10)$$

where $\boldsymbol{\mu}_k$ denotes the vector of the k -th cluster center. Each center $\boldsymbol{\mu}_k$ is computed as a weighted average of the node representations using its corresponding membership probability $\mathcal{Y}_{ik}$ as weights. The final cluster labels c_i are determined by the index of the maximum membership probability.

In weighted graphs, modularity calculation depends not only on node degrees but also requires a comprehensive consideration of the edge weights to reflect the strength of the connections between nodes. To adapt to the dynamically learned graph structure, we replace the original edge weights with a combination of the original edge weights and the learned attention coefficients. This allows for a more accurate modeling of the structural relationships within the current graph. The graph structure optimization between node i and node j is performed as follows:

$$w_{ij} \leftarrow \frac{a'_{ij} + a'_{ji}}{2} \cdot w_{ij}. \quad (11)$$

The modularity-based loss function is expressed as follows:

$$L_M = -\frac{1}{2m} \sum_{i,j} (w_{ij} - \frac{k_i k_j}{2m}) \delta(c_i, c_j), \quad (12)$$

where $m = \frac{1}{2} \sum_{i,j} w_{ij}$ denotes the sum of the weights of all edges, $k_i = \sum_j w_{ij}$ denotes the sum of the weights of all edges connected to the node i , and c_i denotes the i -th cluster, $\delta(c_i, c_j) = 1$ when $c_i = c_j$, otherwise 0.

To enhance the discriminability of node representations, we introduce a loss based on contrastive learning. Specifically, for each target node i , we select its neighbors as positive samples to reconstruct the graph structure, while selecting unrelated nodes as negative samples. The graph structure-based loss function of node i can be expressed as:

$$L_i = \log(\sigma(\mathbf{h}'_i{}^\top \mathbf{h}'_u)) - Q \times \mathbb{E}_{v \sim P_n(u)} \log(\sigma(-\mathbf{h}'_i{}^\top \mathbf{h}'_v)), \quad (13)$$

where u is the positive sample, P_n is the probability distribution of negative samples, Q is the number of negative samples, and $\sigma(\cdot)$ denotes the sigmoid function.

The overall graph structure-based loss is defined as the average of individual node-level losses across the entire graph, formulated as $L_G = \frac{1}{n} \sum_{i=1}^n L_i$.

The final optimization objective function combines the above two sub-loss functions in a weighted manner:

$$L = L_G + \eta L_M. \quad (14)$$

where η is a hyperparameter.

All parameters are learned based on the subgraph. After training is complete, the original graph is sent to the trained model (without passing through cluster-oriented graph contraction module) to obtain clustering results. The entire procedure is shown in Algorithm 1.

5 Experiments

Datasets. The proposed model in this paper is evaluated on three datasets, and the statistical information for each dataset is presented

Table 1: Dataset statistics.

Dataset	Nodes	Edges	Clusters	Density
Vessel01	818	144,898	4	0.434
Vessel10	763	146,981	4	0.505
ML100k	1,612	58,424	9	0.045

Table 2: Clustering performance on five datasets. The bold and underlined values indicate the best and the runner-up results.

Model	Vessel01		Vessel10		ML100K _{N=0%}		ML100K _{N=10%}		ML100K _{N=20%}	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1
SSGCN	43.15	29.49	44.88	26.25	37.66	16.57	36.48	15.15	36.41	14.43
DyFSS	53.06	30.50	52.93	30.44	24.38	13.54	23.88	13.07	23.70	12.87
DGCLUSTER	58.67	57.58	51.11	56.60	36.84	32.85	36.35	37.14	35.85	35.28
SCGC	56.50	35.43	48.62	26.90	44.57	17.38	44.04	14.00	<u>43.73</u>	14.06
DAGC	50.93	32.02	59.13	23.26	33.91	10.96	21.99	12.32	21.54	11.13
GC-SEE	38.84	23.91	32.06	21.17	36.43	31.36	33.96	25.39	33.17	22.14
VGAER	32.32	34.43	33.95	36.46	43.44	<u>48.48</u>	36.85	<u>42.32</u>	37.21	32.58
SUBLIME	31.73	20.49	32.01	21.17	20.83	12.39	22.05	11.69	22.02	12.64
NMFGAAE	36.35	28.93	30.52	24.38	35.64	40.19	31.71	35.38	34.06	38.73
CaEGCN	58.28	36.38	53.43	35.70	44.21	45.43	44.19	44.71	43.15	<u>44.52</u>
GMIM	<u>78.61</u>	<u>69.51</u>	<u>74.84</u>	<u>65.75</u>	<u>44.78</u>	31.17	<u>44.75</u>	28.17	43.79	29.81
CeeGCN	78.97	88.25	75.36	85.94	46.36	63.35	44.97	41.03	43.64	60.76

in Table 1. Node features only include node ID. If other features were available, they could also be included.

Vessel01 and Vessel10 are constructed from the vessel trajectory data from four cities, including timestamps, longitude and latitude positions, and heading direction.

- When the spherical distance between two vessels is less than 1.11 kilometers, they are considered to meet, creating an edge with a weight increment of one.
- As vessels belonging to the same city frequently encounter each other, we use the city to which a vessel belongs as the ground label.
- Each dataset represents the encounters of valid vessel trajectories for a specified month.

ML100K is constructed from the dataset MovieLens 100K. We will include this content in the Supplementary Material. The dataset includes multiple files, with our main focus being on u.data and u.item. (We will release this dataset.)

- Constructing a graph from rating records of users in u.data. First, filter the rating records by each user and sort them by timestamp. Then, compute the edge weights based on consecutive movie ratings. If two movies are rated consecutively by the same user, the edge weight between these two movies is increased by 1.
- Assign labels to movies based on genres in u.item. Since movies may belong to multiple genres, we calculate the frequency of each genre across the dataset and assign the label based on the genre with the highest frequency for each movie.

Methods for Comparison. We compare our proposed method with eleven competitive methods to evaluate the effectiveness of our method, including SSGCN [14], DyFSS [43], DGCLUSTER [4], SCGC [20], DAGC [25], GC-SEE [10], VGAER [27], SUBLIME [23], NMFGAAE [13], CaEGCN [15], and GMIM [1].

Evaluation Metrics and Experimental Setting. We select two widely used metrics to evaluate the clustering results: Accuracy (ACC) and Micro F1-score (F1). We multiply the metrics by 100 for better readability. Our method is implemented with the PyTorch

2.3.0¹. For baselines, we download the source codes and report the corresponding clustering results using the original paper’s settings. For our proposed method, the number of heads is set as 8, and the number of EWSGAT layers is set to 3. We tune “ ϵ ” in the calculation of comprehensive scores, “ α ” in the α -entmax function, “ η ” in the final loss function, the number of EWSGAT layers from {0.0, 0.3, 0.5, 0.8, 1.0}, {1.2, 1.3, 1.4, 1.45, 1.5, 1.55, 1.6, 1.7, 1.8, 1.9}, {0.02, 0.025, 0.03, 0.035, 0.04, 0.06, 0.08}, {2, 3, 4, 5, 6}, respectively. The Adam optimizer is employed with a learning rate of 0.005.

5.1 Clustering Performance Comparisons

The first two datasets are noisy due to human interventions in their construction. In contrast, the third dataset has minimal human intervention. We introduce random noise edges into the low-intervention dataset in different proportions denoted by the variable “N”. We randomly add 10% and 20% of edges to ML100k to create new experimental datasets.

For all methods, we repeat each experiment 5 times by changing the random seeds and report the average results. Table 2 presents a comparison of the clustering performance between CeeGCN and five mainstream graph clustering methods across multiple datasets.

Overall, CeeGCN demonstrates superior clustering capability in all datasets. Notably, the performance of CeeGCN on the vessel datasets significantly surpasses that on ML100k. This is attributed to CeeGCN’s effective implementation of a sparsity-aware method, which reduces computational complexity and noise interference in dense graphs, thereby improving clustering performance. Furthermore, CeeGCN consistently performs well on ML100k, showcasing its ability to accurately identify and handle noisy edges. In particular, introducing 10% and 20% noisy edges leads to varying degrees of performance degradation across all clustering methods, with the F1 showing a particularly marked decline. This indicates that noisy edges weaken the separability of inter-cluster boundaries. Except for SSGCN, the comparative methods assume the reliability of the graph structure, thereby underperforming our approach when considering both metrics. Although SSGCN adopts graph structure learning, it fails to jointly optimize node representations

¹<https://github.com/HaobingLiu/CeeGCN>

Table 3: Ablation study of key designs in CeeGCN.

Model	Vessel01		Vessel10		ML100K	
	ACC	F1	ACC	F1	ACC	F1
CeeGCN-CGC	78.42	87.90	74.59	85.45	43.63	60.76
CeeGCN-EWSGAT	53.67	44.23	53.19	37.86	35.74	42.13
CeeGCN-entmax	75.79	81.08	70.90	76.29	44.85	61.30
CeeGCN- f_{iz}	69.68	72.39	73.65	81.25	44.72	61.54
CeeGCN-EWO	77.14	85.06	75.09	84.59	44.97	62.04
CeeGCN w/o Contraction	78.23	87.14	75.81	84.94	44.96	61.27
CeeGCN	78.97	88.25	75.36	85.94	46.36	63.35

Table 4: The training time (in seconds) of CeeGCN with and without the contraction module.

Dataset	CeeGCN w/o Contraction	CeeGCN
Vessel01	11,846.10	2,576.40
Vessel10	17,767.93	3,701.65
ML100k	8,492.50	994.30

and clustering, thus performing not well. By fusing edge weights in attention coefficient computation and incorporating the α -entmax function to suppress noisy connections, our model can focus on related neighbors, thereby performing well.

5.2 Ablation Study

To prove the effectiveness of the key designs in CeeGCN, we consider different model variants from four perspectives:

- Effect of cluster-oriented graph contraction module. We replace cluster-oriented graph contraction with random sampling to select nodes of similar size, retaining all connecting edges between the selected nodes to obtain a subgraph. We refer to this variant as CeeGCN-CGC.
- Effect of EWSGAT. We replace EWSGAT with vanilla GAT and refer to this variant as CeeGCN-EWSGAT.
- Effect of the α -entmax function. We replace the α -entmax function with the softmax function and refer to this variant as CeeGCN-entmax.
- Effect of the impact factor derived from edge weights. We remove the f_{iz} to observe the changes in clustering results and refer to this variant as CeeGCN- f_{iz} .
- Effect of edge weight optimization. We utilize the original edge weights w_{ij} to get the modularity loss. We refer to this variant as CeeGCN-EWO.
- Effect of the graph contraction. We remove the graph contraction module to observe the changes in clustering performance. We refer to this variant as CeeGCN w/o Contraction.

Table 3 reports the results. CeeGCN-CGC performs worse than CeeGCN. This indicates that randomly sampling nodes fails to effectively preserve the original graph’s clustering structure, whereas the cluster-oriented graph contraction module can achieve this. CeeGCN-EWSGAT performs far worse than CeeGCN. This is because, for graphs with noisy edges, vanilla GAT lacks the ability

to block unrelated neighbors from participating in message propagation, leading to a substantial drop in clustering performance. CeeGCN-EWO performs worse than CeeGCN. This indicates that CeeGCN can better optimize edge weights based on feedback during the training process, while fixing edge weights restricts edge weight optimization. The performance degradation of CeeGCN-entmax is attributed to the non-zero weights assigned by the softmax function, which leads to noise accumulation. In contrast, α -entmax truncates low-correlation weights to zero through an automatic sparsification mechanism. This effectively blocks noise propagation and focuses on key neighbors. The performance degradation of CeeGCN w/o Weight validates the importance of edge weights as prior structural information. Removing f_{iz} leads to the loss of connection strength information, whereas retaining this factor enables EWSGAT to integrate feature similarity with topological strength to guide the clustering task.

By comparing CeeGCN w/o Contraction and CeeGCN, we find that CeeGCN performs better than CeeGCN w/o Contraction in most cases. This may be because the original graph contains more noisy edges, whereas the subgraph consists of important nodes and fewer noisy edges. Training based on the subgraph enables the model to more easily acquire the ability to identify noisy edges. Assuming E is the number of edges, d is the hidden dimension of a single attention head, and there are H total heads per layer, the time complexity of each vanilla GAT layer is $O(E \cdot d \cdot H)$; while the time complexity of each EWSGAT layer is $O(E \cdot (d + 1) \cdot H)$. Compared to unweighted graphs, storing the graph structure of weighted graphs in the form of tuples leads to a 50% increase in GPU memory usage during the model training process. Therefore, graph Contracting is highly necessary.

We further compare the training time of CeeGCN with and without the contraction module. As shown in Table 4, we find that incorporating the contraction module significantly improves the training efficiency. Meanwhile, the contraction module effectively reduces memory usage. Taking the Vessel01 dataset as an example: The original graph includes 818 nodes and 144,898 edges. Storing the graph structure requires approximately 1.74 MB. The extracted subgraph includes ~ 425 nodes and ~ 57183 edges. Storing these edges requires approximately 0.69 MB.

5.3 Hyperparameter Analysis

We select four important hyperparameters to study, including “ ϵ ” in the calculation of comprehensive scores, “ α ” in the “ α -entmax” function, “ η ” in the final loss function, and the calculation method of the self-loop weight.

Figure 3 presents the sensitivity analysis of the value of “ ϵ ”. Experiment results indicate that the model achieves optimal performance when “ ϵ ” is set to 0.5. This phenomenon suggests that both node density and distance information are of equal importance. In Figure 4(a), optimal performance is achieved when “ α ” is set to 1.55. When the “ α ” < 1.55 , the α -entmax function behaves more like softmax function, focusing on all neighbors, which may incorporate unrelated nodes during message passing. However, when the “ α ” > 1.55 , it might overly suppress edge weight information, leading to performance degradation. In Figure 4(b), optimal performance is

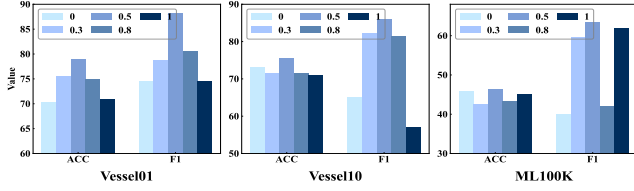
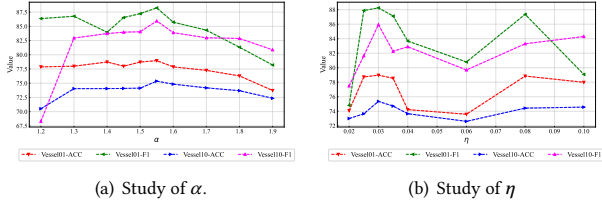
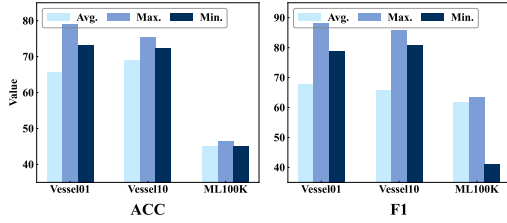
Figure 3: Analysis of ϵ on model performance.Figure 4: Analysis of α and η on model performance.

Figure 5: Clustering results under different self-loop weight calculation methods.

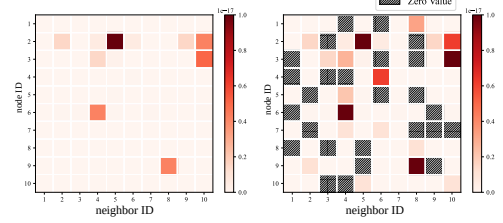
achieved when “ η ” is set to 0.03. This indicates the graph structure-based loss plays a more significant role. As shown in Figure 5, we assign different weight strategies to the self-connected edges of the nodes. These strategies included the average, maximum and minimum values of the weights between a node and its neighbors. Employing the maximum value as the self-loop weight strategy generally produces the best performance across all datasets. By setting the self-weight to the maximum value, we strengthen the node’s focus on its own information.

5.4 Case Study

To validate CeeGCN’s robustness to noisy edges in graph clustering, we randomly select 10 nodes from the ML100K_{N=10%} dataset, each having 10 neighboring nodes.

We use CeeGCN w/o EWSGAT and CeeGCN to compute inter-node attention coefficients, and then visualize the results using heatmaps, as shown in Figure 6. Darker colors indicate a tighter relationship between nodes. The y-axis lists the node IDs, and the x-axis represents the neighbors of node, ordered by ID in ascending order.

In Figure 6(a), we observe that node 3 has strong attention connections with its 4th and 10th neighbors, indicating a close relationship. In Figure 6(b), we find that the attention coefficients between node 3 and three of its neighbors (nodes 1, 6, and 8) are set to 0 (highlighted by shaded blocks), identifying these edges as noisy edges. In contrast, the connections with the 4th and 10th neighbors



(a) CeeGCN-EWSGAT

(b) CeeGCN

Figure 6: Distributions of attention coefficients.
Table 5: Average normalized attention coefficients.

Group	Noiseless edge	Noisy edge
Group 1	0.050	0.015
Group 2	0.029	0.007
Group 3	0.024	0.005

show noticeably deeper colors, signifying their increased importance and relevance. This shows that CeeGCN effectively prioritizes neighbors with stronger relationships while ignoring noisy edges.

To further analyze the effectiveness of noisy edge handling, we conduct a detailed comparison of the average attention coefficients assigned to noisy and noiseless edges. Since both edge weights and the number of neighbors influence attention coefficient computation, we filter the dataset to include 604 nodes with an average edge weight greater than 1.05. The 604 nodes are grouped by their number of neighbors: Group 1 ($0 < \#neighbors \leq 25$), Group 2 ($26 < \#neighbors \leq 50$), and Group 3 ($51 < \#neighbors \leq 100$). Table 5 presents the distribution of attention coefficients within these groups.

Table 5 reveals that neighboring nodes connected by noisy edges consistently receive lower attention coefficients, especially when the number of a node’s neighbors is small, indicating a more pronounced impact of noisy edges in such cases. As the number of neighbors increases, CeeGCN becomes more accurate at identifying important nodes, thereby better suppressing the influence of noisy edges. This demonstrates the effectiveness of combining edge weight information with node features, with model performance significantly improving as the number of neighbors grows.

6 Conclusion

In this paper, we propose a weighted graph clustering model. We design a cluster-oriented graph contraction module. The module captures both intra-cluster and inter-cluster structural characteristics, significantly reducing the graph scale by sampling important nodes. We design an edge-weight-aware sparse graph attention network. The network calculates attention coefficients by considering both node features and edge weights. Noise edges are suppressed via the α -entmax function. We will explore the scalability of our model on larger-scale weighted graphs in the future.

Acknowledgments

This research is supported in part by the National Science Foundation of China (No. 62302469, No. 62176243), the Natural Science Foundation of Shandong Province (ZR2023QF100, ZR2022QF050, ZR2025QB43), and the Fundamental Research Funds for the Central Universities (202513026).

References

- [1] Maedeh Ahmadi, Mehran Safayani, and Abdolreza Mirzaei. 2024. Deep graph clustering via mutual information maximization and mixture model. *KAIS* 66 (2024), 4549–4572.
- [2] Muhammed Fatih Balin and Ümit CATALYÜREK. 2023. Layer-Neighbor Sampling–Defusing Neighborhood Explosion in GNNs. In *NeurIPS*. 25819–25836.
- [3] James C Bezdek. 1981. Objective function clustering. In *Pattern recognition with fuzzy objective function algorithms*. 43–93.
- [4] Aritra Bhowmick, Mert Kosan, Zexi Huang, Ambuj Singh, and Sourav Medya. 2024. DGCLUSTER: A neural framework for attributed graph clustering via modularity maximization. In *AAAI*. 11069–11077.
- [5] Mathieu Blondel, André F. T. Martins, and Vlad Niculae. 2019. Learning classifiers with fenchel-young losses: Generalized entropies, margins, and algorithms. In *AISTATS*. 606–615.
- [6] Deyu Bo, Xiao Wang, Chuan Shi, Meiqi Zhu, Emiao Lu, and Peng Cui. 2020. Structural Deep Clustering Network. In *WWW*.
- [7] Karima Boutalbi, Rafika Boutalbi, Hervé Verjus, and Kavé Salamatian. 2024. Hierarchical Tensor Clustering for Multiple Graphs Representation. In *WWW*.
- [8] Xiao Chunjing, Ranhao Guo, Zhang Yongwang, and Xiaoming Wu. 2025. Learning Multi-interest Embedding with Dynamic Graph Cluster for Sequence Recommendation. In *UAI*. 4652–4662.
- [9] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. 2016. Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). In *ICLR*.
- [10] Shifei Ding, Benyu Wu, Xiao Xu, Lili Guo, and Ling Ding. 2023. Graph clustering network with structure embedding enhanced. *PR* 144 (2023).
- [11] Manish Gupta, Amit Pathak, and Soumen Chakrabarti. 2008. Fast algorithms for topk personalized pagerank queries. In *ACM WWW*. 1225–1226.
- [12] Jie Hao and William Zhu. 2023. Deep graph clustering with enhanced feature representations for community detection. *Applied Intelligence* 53 (2023), 1336–1349.
- [13] Chaobo He, Yulong Zheng, Xiang Fei, Hanchao Li, Zeng Hu, and Yong Tang. 2021. Boosting nonnegative matrix factorization based community detection with graph attention auto-encoder. *TBD* 8 (2021), 968–981.
- [14] Xiaxia He, Boyue Wang, Ruikun Li, Junbin Gao, Yongli Hu, Guangyu Huo, and Baocai Yin. 2023. Graph structure learning layer and its graph convolution clustering application. *Neural Networks* 165 (2023), 1010–1020.
- [15] Guangyu Huo, Yong Zhang, Junbin Gao, Boyue Wang, Yongli Hu, and Baocai Yin. 2021. CaEGCN: Cross-attention fusion based enhanced graph convolutional network for clustering. *TKDE* 35 (2021), 3471–3483.
- [16] Guangyu Huo, Yong Zhang, Junbin Gao, Boyue Wang, Yongli Hu, and Baocai Yin. 2023. CaEGCN: Cross-Attention Fusion Based Enhanced Graph Convolutional Network for Clustering. *TKDE* 35 (2023), 3471–3483.
- [17] Di Jin, Ziyang Liu, Weihao Li, Dongxiao He, and Weixiong Zhang. 2019. Graph convolutional networks meet markov random fields: Semi-supervised community detection in attribute networks. In *AAAI*. 152–159.
- [18] Zhao Kang, Xuanting Xie, Bingheng Li, and Erlin Pan. 2025. CDC: A Simple Framework for Complex Data Clustering. *IEEE Transactions on Neural Networks and Learning Systems* 36 (2025), 13177–13188.
- [19] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*.
- [20] Gayan K Kulatilake, Marius Portmann, and Shekhar S Chandra. 2025. SCGC: Self-supervised contrastive graph clustering. *Neurocomputing* 611 (2025), 128629.
- [21] Meng Liu, Yue Liu, Ke Liang, Wenxuan Tu, Siwei Wang, Sihang Zhou, and Xinwang Liu. 2024. Deep Temporal Graph Clustering. In *ICLR*.
- [22] Yue Liu, Xihong Yang, Sihang Zhou, Xinwang Liu, Zhen Wang, Ke Liang, Wenxuan Tu, Liang Li, Jingcan Duan, and Cancan Chen. 2023. Hard sample aware network for contrastive deep graph clustering. In *AAAI*. 8914–8922.
- [23] Yixin Liu, Yu Zheng, Daokun Zhang, Hongxu Chen, Hao Peng, and Shirui Pan. 2022. Towards unsupervised deep graph structure learning. In *ACM WWW*. 1392–1403.
- [24] Erlin Pan and Zhao Kang. 2023. Beyond Homophily: Reconstructing Structure for Graph-agnostic Clustering. In *ICML*.
- [25] Zhihao Peng, Hui Liu, Yuheng Jia, and Junhui Hou. 2023. Deep attention-guided graph clustering with dual self-supervision. *IEEE Transactions on Circuits and Systems for Video Technology* 33 (2023), 3296–3307.
- [26] Ben Peters, Vlad Niculae, and André F. T. Martins. 2019. Sparse Sequence-to-Sequence Models. In *ACL*. 1504–1519.
- [27] Chenyang Qiu, Zhaoci Huang, Wenzhe Xu, and Huijia Li. 2022. VGAER: graph neural network reconstruction based community detection. *Association for the Advancement of Artificial Intelligence*, (2022).
- [28] Zhixiang Shen and Zhao Kang. 2025. When Heterophily Meets Heterogeneous Graphs: Latent Graphs Guided Unsupervised Representation Learning. *IEEE Transactions on Neural Networks and Learning Systems* 36 (2025), 10283–10296.
- [29] Seiyun Shin, Ilan Shomorony, and Han Zhao. 2023. Efficient learning of linear graph neural networks via node subsampling. In *NeurIPS*. 55479–55501.
- [30] Ippatapu Venkata Srichandra and Pratiti Bhadra. 2024. Community Detection Using Graph Attention Networks Clustering Algorithm. In *IEEE International Conference for Convergence in Technology*. 1–6.
- [31] Qingyun Sun, Jianxin Li, Hao Peng, Jia Wu, Xingcheng Fu, Cheng Ji, and S Yu Philip. 2022. Graph structure learning with variational information bottleneck. In *AAAI*. 4165–4174.
- [32] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *ICLR*.
- [33] Chun Wang, Shirui Pan, P Yu Celina, Ruiqi Hu, Guodong Long, and Chengqi Zhang. 2022. Deep neighbor-aware embedding for node clustering in attributed graphs. *PR* 122 (2022), 108230.
- [34] Ruijia Wang, Shuai Mou, Xiao Wang, Wanpeng Xiao, Qi Ju, Chuan Shi, and Xing Xie. 2021. Graph structure estimation neural networks. In *ACM MM*. 342–353.
- [35] Tong Wang, Junhua Wu, Yaolei Qi, Xiaoming Qi, Juwei Guan, Yuan Zhang, and Guanyu Yang. 2023. Neighborhood contrastive representation learning for attributed graph clustering. *Neurocomputing* 562 (2023), 126880.
- [36] Yi Wen, Suyuan Liu, Xinhang Wan, Siwei Wang, Ke Liang, Xinwang Liu, Xihong Yang, and Pei Zhang. 2023. Efficient multi-view graph clustering with local and global structure preservation. In *ACM MM*. 3021–3030.
- [37] Xihong Yang, Yue Liu, Sihang Zhou, Siwei Wang, Xinwang Liu, and En Zhu. 2022. Contrastive deep graph clustering with learnable augmentation. *arXiv preprint arXiv:2212.03559* abs/2212.03559 (2022).
- [38] Jiaxuan You, Xiaobai Ma, Daisy Yi Ding, Mykel J. Kochenderfer, and Jure Leskovec. 2020. Handling missing data with graph representation learning. In *NeurIPS*. 19075–19087.
- [39] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor K. Prasanna. 2020. Graphsaint: Graph sampling based inductive learning method. In *ICLR*.
- [40] Qi Zhang, Yanfeng Sun, Yongli Hu, Shaofan Wang, and Baocai Yin. 2023. A subgraph sampling method for training large-scale graph convolutional network. *Information Sciences* 649 (2023), 119661.
- [41] Zaixi Zhang, Qi Liu, Qingyong Hu, and Chee-Kong Lee. 2022. Hierarchical graph transformer with adaptive node sampling. In *NeurIPS*.
- [42] Huasong Zhong, Jianlong Wu, Chong Chen, Jianqiang Huang, Minghua Deng, Liqiang Nie, Zhouchen Lin, and Xian-Sheng Hua. 2021. Graph contrastive clustering. In *ICCV*. 9224–9233.
- [43] Pengfei Zhu, Qian Wang, Yu Wang, Jialu Li, and Qinghua Hu. 2024. Every node is different: Dynamically fusing self-supervised tasks for attributed graph clustering. In *AAAI*. 17184–17192.
- [44] Difan Zou, Ziniu Hu, Yewen Wang, Song Jiang, Yizhou Sun, and Quanquan Gu. 2019. Layer-dependent importance sampling for training deep and large graph convolutional networks. In *NeurIPS*. 11247–11256.

A Appendix

A.1 Detailed Calculation of Threshold τ

We provide the detailed calculation process for the threshold τ in Algorithm 1, utilizing the Bisection method to calculate the threshold with high efficiency.

A.2 Details of the Baselines

We compare CeeGCN with several advanced graph clustering methods. The detailed introduction of these methods is as follows:

- **SSGCN** [14]: The method uses a graph structure learning layer and an adaptive graph convolutional layer to jointly optimize graph structure and node representations. By employing sparse representation, it effectively handles data interference, thereby enhancing clustering performance.
- **DyFSS** [43]: The method assigns each self-supervised learning (SSL) task to a task-specific GCN layer to extract features using the corresponding SSL loss.
- **DGCluster** [4]: The method proposes a trainable modular method that models the graph structure using GNN (e.g., GCN, GAT) to learn node representations, thereby accomplishing cluster number estimation.
- **SCGC** [20]: The method based on graph autoencoders, aims to improve the efficiency and scalability of existing deep graph clustering methods by decoupling transformation and propagation operations.
- **DAGC** [25]: This method employs various fusion strategies to integrate feature learning from both GCN and autoencoders, along with clustering distribution information, dynamically weighting their significance using attention mechanisms.
- **GC-SEE** [10]: This model utilizes autoencoders for attribute feature extraction, graph attention autoencoders to learn

node importance and structural information, adaptive convolution for fusing attribute and structural information.

- **VGAER** [27]: The technique of graph variational self-encoders enables the reconstruction of non-linear modularity by integrating modularity information and graph structure.
- **SUBLIME** [23]: The method effectively learns the graph structure information by combining the learned adjacency matrix with data features and maximizing view consistency through node-level contrastive learning.
- **NMF-GAAE** [13]: The graph attention auto-encoder is utilized to learn the representation of nodes, and the cluster structure is revealed through a non-negative matrix decomposition of the node representation.
- **CaEGCN** [15]: This model uses an attention mechanism to combine the distinct representations learned by the content self-encoder and graph self-encoder modules, which capture heterogeneous features.
- **GMIM** [1]: The method proposes a contrastive learning framework that aims to directly optimize node embeddings beneficial for clustering. It introduces a gaussian mixture distribution in the embedding space, thereby promoting the generation of node representations suitable for the clustering task. GMIM also incorporates a graph diffusion mechanism to capture the global structural information of the graph.

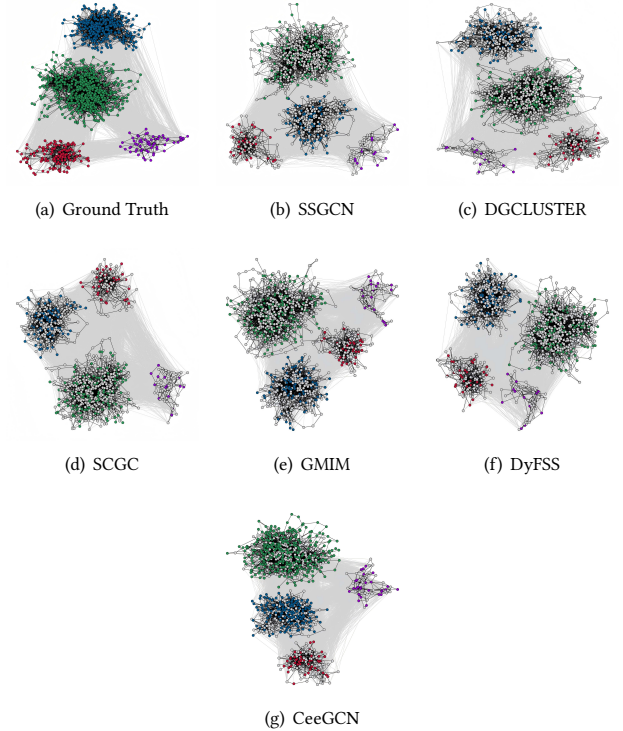


Figure 1: Two-dimensional visualization of clustering results on Vessel01.

Algorithm 1: Calculation of τ by Bisection

Input : e'_i : the redefined attention coefficients between node i and its neighbors; d : dimension of the vector e'_i ; α : the hyperparameter of the α -entmax function.

Output : τ : threshold.

Define : $p = [(\alpha - 1)e'_{iz} - \tau(e'_{iz})]_+^{\frac{1}{\alpha-1}}$;

$\tau_{\min} = \max(e'_i) - 1$;

$\tau_{\max} = \max(e'_i) - d^{1-\alpha}$;

```

1 while sum  $\neq$  1 do
2    $\tau = (\tau_{\min} + \tau_{\max})/2$ ;
3   sum =  $\sum_{i=1}^d p_i(\tau)$ ;
4   if sum < 1 then
5      $\tau_{\max} = \tau$ ;
6   else
7      $\tau_{\min} = \tau$ ;
8   end
9 end
10 return  $\tau$ ;
```

A.3 Visualization

To intuitively compare clustering results across different methods, we visualize the Vessel01 dataset with ground truth labels, where each color represents a distinct cluster. Nodes with inconsistent labels are shown in gray. Thus, a higher proportion of gray nodes in the clustering result indicates poorer performance.

The number of gray nodes in Figure 1 highlights that the clustering result obtained by CeeGCN involve fewer gray nodes, indicating more accurate node assignments to clusters. In contrast, models like GMIM show a higher prevalence of gray nodes. This demonstrates that CeeGCN outperforms these models, providing more accurate clustering results that align closely with ground truth labels.